PROJECT DEFENCE BRIEF

AI Virtual Office

A multi-agent business analysis platform — what it does, why it is built this way, and how to defend every decision in it.

FASTAPI + REACT ~7,500 LINES 51 TESTS POSTGRESQL / SQLITE

START HERE ·THE OPENING

What to say first

> IF YOUR TEACHER ASKS "WHAT IS IT?"
>
> “It's a platform where a business owner asks a question in plain language, and a team of specialist AI agents analyses it against that company's *own* documents and metrics — then a strategy agent reconciles their findings into one decision brief. Anything that commits money or headcount waits for human approval.”

That sentence contains the four things that make it a system rather than a chatbot: **routing** (which specialists are relevant), **grounding** (its own data, with citations), **synthesis** (reconciling disagreement), and **a human gate** (approval before action).

If you only remember one thing: the interesting part is not the AI. It is the *pipeline around* the AI.

The problem it addresses

A small company's information is scattered: numbers live in spreadsheets, context lives in documents, and judgement lives in people's heads. When the owner asks “why is our margin falling?”, answering it properly means pulling all three together — and the answer usually needs several departments to weigh in, who often disagree.

This project models that process. Each department becomes an agent with a defined remit; the question is routed to whichever ones are relevant; each answers from the same shared evidence base; and a final agent reconciles them and **names the contradictions** instead of averaging them away.

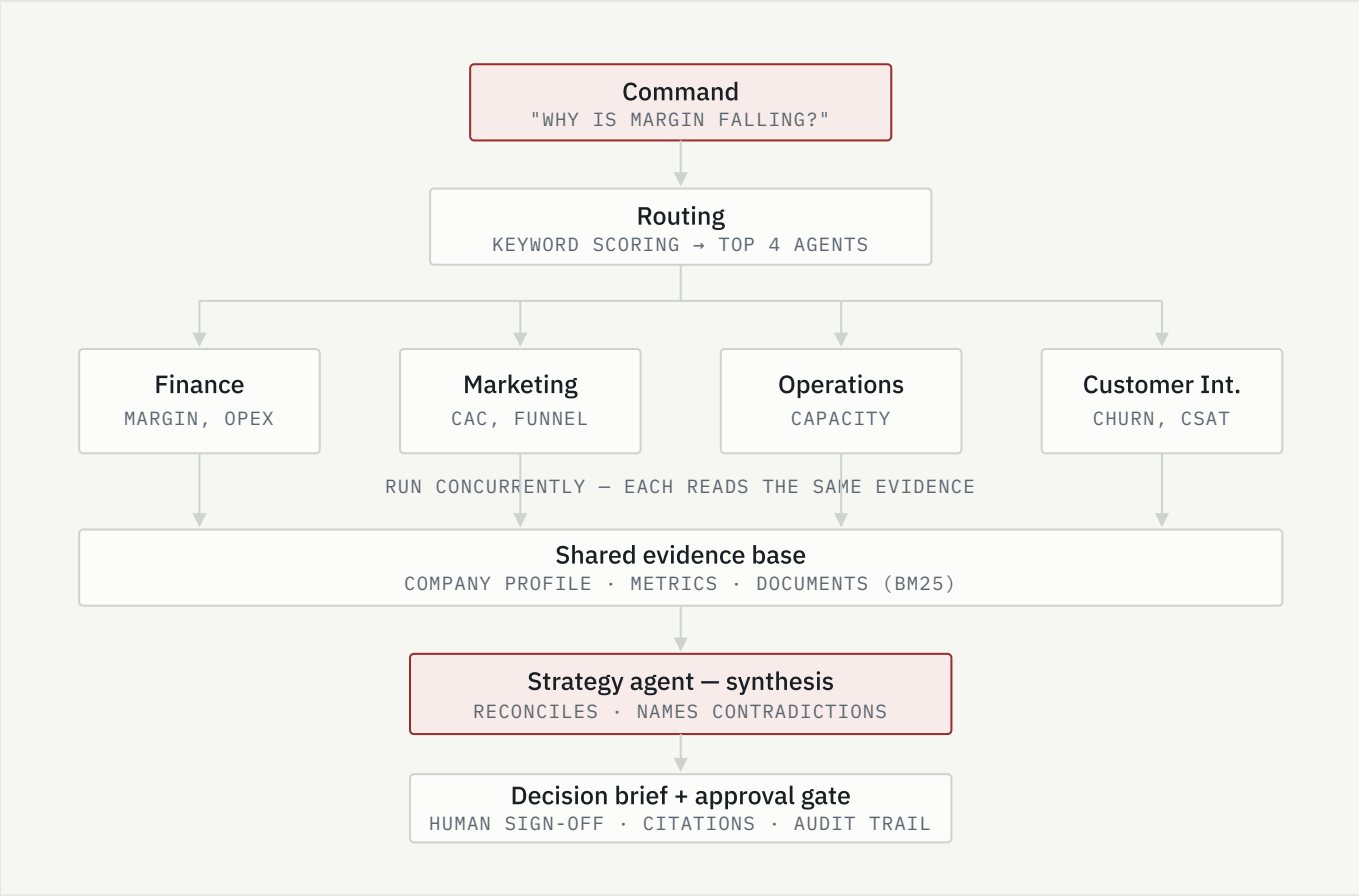
BE HONEST ABOUT SCOPE

Do not claim this replaces a finance team. It is a working model of a decision pipeline, demonstrated on business analysis. Teachers respect a bounded claim far more than an inflated one — and an inflated claim is the easiest thing in the world to puncture with one follow-up question.

MECHANISM ·HOW IT WORKS

What happens when you press Execute

This is the diagram to draw on the whiteboard if you are asked. Every stage is a real function you can point to in the code.



`backend/app/services/orchestrator.py — execute_work_order()`

The part that surprises people

The synthesis step does not blend the findings into a bland average. It actively looks for **contradictions between departments** — for example, Operations reporting 91% capacity while Marketing is asked to generate more demand. It states that conflict plainly, because that conflict *is* the decision the executive has to make.

What it is built from

LAYER	TECHNOLOGY	WHY THIS ONE
API	FastAPI, Pydantic v2	Type-validated requests and auto-generated docs at `/docs`
Database	PostgreSQL or SQLite	12 tables, real foreign keys; one env var switches between them
ORM & schema	SQLAlchemy 2.0, Alembic	Versioned, reversible migrations rather than ad-hoc table creation
Auth	JWT + bcrypt	Four roles, enforced server-side as a dependency
Retrieval	Custom BM25	No vector database needed — works offline and in CI
Reasoning	Anthropic SDK `(optional)`	Structured outputs; falls back to a deterministic engine
Frontend	React 19, TypeScript	Strict mode; charts hand-written in SVG, no chart library
Deployment	Docker Compose, nginx	Database, API and web as one reproducible stack

12	9	7	51
DB TABLES	SCREENS	AGENTS	TESTS

DEPTH · DECISIONS

Design decisions you can defend

This section matters more than the feature list. A teacher tests whether you understand *trade-offs* — so for each decision, know the alternative you rejected and what it cost you.

RETRIEVAL BM25 instead of a vector database

What: Documents are chunked and indexed as term-frequency maps in the database. Search ranks them with BM25, a classic information-retrieval algorithm.

Why: No embedding model to download, no vector database to run, no network call. Search behaves identically offline, in tests, and in production.

Cost: No semantic matching — a search for “staff costs” will not match “payroll expenditure”. Upgrading means adding a vector column and swapping the scoring function; the ingestion pipeline and API stay unchanged.

RESILIENCE The LLM is optional, not required

What: With an API key, agents reason through Claude. Without one, a deterministic engine derives findings from actual metric trends and document passages. Both return the *same* Pydantic types.

Why: Because the return types are identical, the API, the UI and every test are unchanged either way. The fallback is a peer, not a degraded special case. Each work order records which engine ran — `llm`, `heuristic`, or `hybrid` when one agent fell back mid-run.

Cost: Two analysis paths to maintain instead of one.

CORRECTNESS Health scoring has three metric goals

What: Metrics are scored as *higher-is-better* (margin), *lower-is-better* (churn), or *target-band* (capacity utilisation, where both idle and overloaded are unhealthy).

Why: The first version treated every metric as higher-is-better, so 91% capacity utilisation scored as excellent when it actually signals delivery risk. The score was inflated by four points.

Cost: Each metric needs a declared goal and target rather than being scored generically.

HONESTY Confidence is proportional to evidence

What: An agent's confidence rises when the question names a metric it holds and a document corroborates it, and falls when it is reasoning from thin evidence. It is capped below certainty.

Why: A system that reports 95% confidence on everything teaches its user to ignore the number. Confidence has to cost something to be worth anything.

DATA MODEL Agents are database rows, not hardcoded classes

What: Each company gets its own copy of the seven-agent roster, with its own prompts and enable flags.

Why: Different companies can disable agents or customise remits without a code change. Disabled agents are skipped during routing.

Cost: Changing the standard roster requires a backfill across existing companies.

ISOLATION Every query is scoped by company

What: All business tables carry a `company_id`, and every query filters on the caller's company — there is no code path that reads across tenants.

Why: Multi-tenancy enforced at the query layer, not by hiding things in the UI. Tested explicitly: a second company requesting another's work order by ID receives 404, not 403 — which avoids confirming the record exists.

The demonstration, in order

Six minutes, building from visible surface to hidden machinery. Do not open the code first — show it working, then reveal how.

01 **Sign in**

Land on the Command Center. Point out the composite health score and KPI tiles.

> *"Every number on this screen is computed from stored metrics. Nothing here is hardcoded."*

02 **Start typing a question — pause before running it**

Agent chips appear beneath the input as you type.

> *"Before it runs anything, it shows which specialists it would consult, and whether the result will need approval. That's a separate preview endpoint — routing without execution."*

03 **Execute, then open the result**

Show the three tabs: Executive Brief, Agent Findings, Sources.

> *"Four departments analysed this in parallel, and the strategy agent reconciled them. Note it explicitly says the two departments contradict each other rather than splitting the difference."*

04 **Open the Sources tab**

Real passages retrieved from the indexed document, with relevance scores.

> *"It cites the specific passage it used. If I delete this document, the analysis loses that evidence — it isn't inventing it."*

05 **Ask the same agent a different question**

Go to a workstation and ask two contrasting things — "why did margin fall?" then "what is our runway?"

> *"The question changes which metric leads, which passage is quoted, and how the recommendation is worded. It's answering the question, not reciting a dashboard."*

06 **Upload a document, then search it**

Any PDF or text file. Show the chunk count, then search a phrase from it.

> *"Text extracted, chunked, indexed, and searchable immediately — and now available as evidence to every agent."*

07

Finish on Approvals

Approve something with a comment; show it move into Decision history.

> *“Anything that commits money or headcount stops here for a human. The decision, who made it and when are all recorded in the audit trail.”*

BEFORE YOU PRESENT

Start both servers and log in *before* the room is watching. Have a document ready to upload. If you have an API key, set it so the Settings screen reads “Anthropic” rather than “Deterministic” — and if you don't, that's a feature to point at, not a gap to hide.

Questions you should expect

These are the questions a sharp teacher actually asks. Read the answers until you can give them in your own words — reciting them verbatim is worse than paraphrasing badly.

Why seven agents? Couldn't one AI call do the same thing?

For a question touching one department, yes — and honestly, one call would be cheaper. The structure earns its keep in three places: routing means a question about churn never pays for a marketing analysis; each agent retrieves *its own* context, so the evidence is targeted rather than one giant prompt; and separate findings make contradictions visible, because you can see Operations and Marketing reaching incompatible conclusions from the same data.

If it were one call, the model would smooth that disagreement into a single confident paragraph — which is exactly the failure the human approval gate exists to catch.

How is this different from just asking ChatGPT?

A chat window has no memory of your business, no ability to cite which document a claim came from, no permission model, and no record of who approved what. This is grounded in one company's own data, every claim carries a citation, access is role-controlled, and every meaningful action is written to an audit trail.

Put differently: the model is a component here, not the product. Remove it and the system still runs.

What happens if the API key is missing or the API is down?

It keeps working. A deterministic engine derives findings from real metric trends and retrieved passages, and returns the identical data structures — so nothing downstream can tell the difference structurally. Failure is contained per agent: if one agent's call fails mid-run, only that agent falls back, and the work order is labelled `hybrid` so the record is honest about what actually happened.

This is tested — there are tests that simulate an API outage and assert the work order still completes.

How do you stop one company seeing another's data?

Every business table carries a `company_id`, and the current user's company is resolved from their token on every request. Queries filter on it at the database layer — there is no endpoint that can return another tenant's row.

There are explicit tests for this: a second company requesting another's work order, document, or attempting a delete by ID all receive 404. It returns 404 rather than 403 deliberately, so the response doesn't confirm that the record exists.

Why not use a vector database, like everyone else?

Because it would add a service to run, a model to download, and a network call, in exchange for semantic matching this dataset doesn't yet need. BM25 is a well-established ranking function and it makes the system fully offline-capable — which also means the tests are fast and deterministic.

The upgrade path is deliberately short: add a vector column to the chunks table and replace one scoring function. Ingestion and the API contract are untouched. Choosing the simpler thing *and knowing exactly how to outgrow it* is a stronger answer than adopting infrastructure by default.

What was the hardest bug you dealt with?

The one worth telling is the health score. Capacity utilisation was being scored as higher-is-better, so 91% — which actually means the delivery team is overloaded and missing deadlines — scored as excellent. Fixing it meant recognising that metrics have three different shapes: some you want high, some low, and some inside a band. That changed the score from 78 to a truthful 74.

A second one worth mentioning: adding word-stemming to search so “prices” matches “price”. The subtlety is that stems must *agree* — the base word and its inflections have to land on the same key — and a first attempt had “fall” and “falling” producing different keys. It also invalidated every stored index, which is why there is now a `reindex` command; changing the tokeniser without it would silently break search on existing documents.

How would you scale this?

Three things, in order. First, work orders execute synchronously — fine at the deterministic engine's ~20ms, but LLM calls hold a connection for seconds, so that moves to a task queue. It is already a single self-contained function, so this is a contained change. Second, retrieval currently loads all of a company's chunks per query; it needs a term-based pre-filter or a real index. Third, add rate limiting on authentication.

Knowing where your own bottlenecks are is a better answer than claiming there aren't any.

What doesn't it do well?

The named agent personas are largely presentation — the real work is done by routing, retrieval and synthesis. Business analysis over twenty metrics is also not a problem that genuinely needs multiple agents; the architecture would earn its keep more clearly in a document-heavy domain like due diligence or contract review, where synthesis across many sources actually costs a human hours.

Say this before your teacher does. Knowing the limits of your own design reads as maturity; discovering them live reads as a gap.

Did you use AI tools to build this?

Answer honestly and specifically — vague answers here go badly. Say which parts you designed, which you had assistance with, and what you changed or fixed yourself. Being able to explain *why* each decision was made is what actually demonstrates understanding, and this brief exists to get you there.

Check your institution's policy on AI assistance and disclose accordingly. A disclosed, well-understood project is worth far more than an undisclosed one you cannot defend under questioning.

If asked to open the code

Know these five files. If you can navigate confidently to the right one, it signals you know the codebase.

FILE	WHAT IT HOLDS
`services/orchestrator.py`	The whole pipeline — routing, parallel execution, synthesis, approval
`services/rag.py`	Text extraction, chunking, stemming, BM25 ranking
`services/heuristics.py`	The deterministic engine — how findings are derived without an LLM
`agents/registry.py`	The seven agents, their prompts, keywords and routing logic
`models/entities.py`	All 12 tables and their relationships

Two things worth showing

- **The API documentation at `/docs`** — FastAPI generates it from the type hints. Every endpoint is browsable and testable in the browser.
- **The test suite** — `make test` runs 51 tests in about six seconds. Tests covering tenant isolation and the LLM-outage fallback are the ones worth naming.

Study this until you can explain the pipeline from memory and give three trade-offs in your own words. That is the difference between presenting a project and defending one.